

Especificação das interfaces de todos os módulos e da estrutura da Flow Table e Especificação da lógica de funcionamento e sinais de controle de todos os módulos

1. Objetivo

A etapa de hardware do projeto tem como objetivo desenvolver uma arquitetura em FPGA capaz de receber pacotes de uma comunicação TCP/IP, identificar a qual fluxo cada pacote pertence e atualizar, de forma incremental, as informações necessárias para a posterior extração de features utilizadas por um sistema de detecção de intrusão baseado em Machine Learning (ML).

A ideia é que o processamento ocorra de forma **online**, ou seja, sem a necessidade de armazenar todos os pacotes para realizar posteriormente o processamento em software. A cada pacote recebido, seus metadados são extraídos e utilizados para localizar ou criar uma entrada correspondente ao fluxo na memória da FPGA.

A arquitetura proposta é composta por duas etapas principais:

Etapa	Função
Extração de metadados	Identificar as informações necessárias do pacote, como endereços IP, portas, protocolo, tamanho e timestamp
Gerenciamento do fluxo e extração de features	Associar o pacote a um fluxo, manter seu estado e atualizar as estatísticas acumuladas

Essa abordagem é baseada em conceitos utilizados em trabalhos de monitoramento de fluxos implementados em FPGA. Sha et al. apresentam, por exemplo, uma arquitetura de monitoramento de fluxos que mantém uma tabela de estados e realiza atualizações incrementais das informações associadas a cada fluxo, utilizando FPGA para processamento de tráfego em alta velocidade.

2. Identificação de um fluxo

Para determinar a qual fluxo um pacote pertence, será utilizada a **5-tupla**, composta por:

- endereço IP de origem;
- endereço IP de destino;
- porta de origem;
- porta de destino;
- protocolo de transporte.

A chave do fluxo será representada por:

Flow Key = Src IP || Dst IP || Src Port || Dst Port || Protocol

Considerando IPv4, a largura da chave será:

Campo	Largura
Source IP	32 bits
Destination IP	32 bits
Source Port	16 bits
Destination Port	16 bits
Protocol	8 bits
Total	104 bits

A utilização de uma chave de **104 bits** também aparece em trabalhos de monitoramento de fluxos em FPGA, como Sha et al. e Nguyen et al., que utilizam a 5-tupla IPv4 como identificador do fluxo.

Nesta primeira implementação será considerado um **uniflow**, isto é, a direção da comunicação faz parte da identificação. Portanto, um pacote de A para B e um pacote de B para A serão considerados fluxos diferentes.

3. Arquitetura dos módulos

A arquitetura será dividida em módulos especializados, de forma que cada etapa tenha uma função bem definida.

Módulo	Responsabilidade
Packet Parser	Receber o pacote e extrair os campos necessários
Flow Identification	Formar a Flow Key a partir dos campos extraídos
Hash Unit	Transformar a Flow Key em uma posição inicial da Flow Table
Flow Table Controller	Controlar a consulta e determinar HIT, MISS ou COLLISION
Flow Table	Armazenar o estado dos fluxos
Feature Update Unit	Atualizar as estatísticas de um fluxo existente
Flow Finalizer	Calcular as features finais e gerar o vetor de saída

A separação entre identificação do fluxo, armazenamento do estado e atualização das estatísticas é coerente com arquiteturas de monitoramento de fluxos em FPGA. Em Sha et al., por exemplo, a arquitetura apresenta componentes responsáveis pelo gerenciamento das entradas e pela atualização dos valores associados aos fluxos.

4. Packet Parser

O **Packet Parser** será responsável por analisar o pacote recebido e disponibilizar os metadados necessários para as etapas seguintes.

Nesta versão do projeto serão considerados:

- Ethernet II;
- IPv4;
- TCP;
- UDP.

Ao final do parsing, o módulo deverá disponibilizar, no mínimo:

Informação	Largura
Source IP	32 bits
Destination IP	32 bits
Source Port	16 bits
Destination Port	16 bits
Protocol	8 bits
Packet Length	16 bits
Timestamp	64 bits

A utilização de uma arquitetura modular para parsing de cabeçalhos é comum em implementações de processamento de pacotes em FPGA. Kekely, Puš e Kořenek, por exemplo, apresentam um parser modular e pipeline para processamento de cabeçalhos de pacotes em FPGA.

5. Packet Length

O campo **Packet Length** será representado inicialmente por **16 bits**.

Essa escolha está relacionada à própria representação do campo *Total Length* do cabeçalho IPv4, que possui 16 bits. Dessa forma, a arquitetura consegue representar diretamente o tamanho de um pacote IPv4.

É importante diferenciar o tamanho individual do pacote da quantidade total de bytes do fluxo:

- packet_length: tamanho do pacote atual → **16 bits**;
- byte_count: quantidade acumulada de bytes do fluxo → **40 bits**.

Por exemplo, se um fluxo receber três pacotes de 100, 500 e 200 bytes:

byte_count = 100 + 500 + 200 = 800 bytes

O packet_length continua sendo apenas o tamanho do pacote que está sendo processado naquele momento.

6. Hash Unit

Depois que a Flow Key é formada, ela será utilizada pela **Hash Unit** para determinar uma posição inicial na Flow Table.

A função hash recebe os 104 bits da Flow Key e produz um índice correspondente à posição candidata na tabela.

Para a primeira implementação será utilizada uma Flow Table com **32 entradas**. Portanto:

32 entradas → 5 bits de endereço

Assim:

Informação	Largura
Flow Key	104 bits
Hash Index	5 bits

Número de entradas 32

O algoritmo de hash específico poderá ser definido durante a implementação e avaliação da arquitetura. Neste estágio, o ponto principal é estabelecer a interface entre a identificação do fluxo e a localização de sua entrada na memória.

A utilização de funções hash para localizar fluxos em estruturas de armazenamento em FPGA é apresentada, por exemplo, em trabalhos de monitoramento de fluxos utilizando tabelas hash.

7. Flow Table

A **Flow Table** é o elemento central responsável por armazenar o estado dos fluxos que estão sendo monitorados.

Cada posição da tabela representa um fluxo e mantém as informações necessárias para atualizar as features à medida que novos pacotes são recebidos.

Para a primeira implementação, será utilizada uma tabela com **32 entradas**, escolhida como uma configuração inicial reduzida para facilitar a simulação e validação da arquitetura. Em uma implementação posterior, esse número poderá ser aumentado de acordo com a quantidade de recursos disponíveis na FPGA e com os requisitos de desempenho.

A estrutura proposta é:

Campo	Largura
Valid	1 bit

Campo	Largura
Flow Key	104 bits
Packet Count	24 bits
Byte Count	40 bits
First Timestamp	64 bits
Last Timestamp	64 bits
Total	297 bits

Portanto, cada registro da Flow Table possui **297 bits**.

Para 32 entradas:

$$32 \times 297 = 9504 \text{ bits}$$

Essa estrutura foi definida tendo como referência principalmente a organização apresentada por Sha et al., que utiliza uma chave de fluxo de 104 bits e mantém informações como timestamps, contagem de pacotes e tamanho acumulado do fluxo. Entretanto, o número de entradas e a configuração da tabela propostos aqui são **decisões específicas deste projeto para a primeira versão**, e não valores impostos pelo artigo.

8. Campo Valid

O campo valid indica se determinada posição da Flow Table está sendo utilizada.

Valid	Significado
0	Entrada livre
1	Entrada ocupada por um fluxo válido

Esse campo permite diferenciar uma posição ainda não utilizada de uma posição que contém informações de um fluxo.

9. Packet Count

O packet_count representa a quantidade de pacotes que já foram associados ao fluxo.

Quando um novo fluxo é criado:

$$\text{packet_count} = 1$$

A cada novo pacote pertencente ao mesmo fluxo:

$$\text{packet_count} = \text{packet_count} + 1$$

A largura escolhida para a primeira implementação é de **24 bits**.

Essa representação também é utilizada na estrutura de estado apresentada por Sha et al., sendo, portanto, uma referência adequada para a definição inicial da arquitetura.

10. Byte Count

O `byte_count` representa a quantidade total de bytes acumulados pelo fluxo.

Quando o fluxo é criado:

`byte_count = packet_length`

A cada novo pacote:

`byte_count = byte_count + packet_length`

Foi definida uma largura de **40 bits** para esse campo.

Essa largura é significativamente maior que os 16 bits utilizados para representar o tamanho de um pacote individual, pois o objetivo é permitir o acúmulo de dados de vários pacotes durante a existência do fluxo.

A largura de 40 bits também é utilizada por Sha et al. para representar o tamanho acumulado do fluxo.

11. Timestamps

Cada entrada possui dois timestamps:

- `first_timestamp`: instante de chegada do primeiro pacote do fluxo;
- `last_timestamp`: instante de chegada do pacote mais recente.

Ambos serão representados por **64 bits**.

Na criação de um fluxo:

`first_timestamp = timestamp`

`last_timestamp = timestamp`

Nos pacotes seguintes:

`first_timestamp` permanece inalterado

`last_timestamp = timestamp atual`

O timestamp será fornecido inicialmente pelo ambiente de teste a partir das informações disponíveis no PCAP. Para a implementação em hardware, ele deverá ser representado como um valor inteiro com resolução definida para todo o sistema, evitando a utilização de ponto flutuante.

12. Flow Table Controller

O **Flow Table Controller** será responsável por coordenar o processamento das entradas e controlar a comunicação entre os módulos.

Depois que o Packet Parser e o Flow Identification disponibilizam os metadados, o Controller solicita uma consulta à Flow Table utilizando o índice produzido pela Hash Unit.

A consulta poderá resultar em três situações:

Resultado	Condição	Ação
MISS	Entrada livre	Criar novo fluxo
HIT	Entrada contém a mesma Flow Key	Atualizar fluxo
COLLISION	Entrada ocupada por outra Flow Key	Sinalizar colisão

O Controller é importante porque centraliza a decisão sobre o que deve acontecer após cada consulta, evitando que os módulos de armazenamento e atualização precisem controlar diretamente todo o fluxo de operação.

13. Operação MISS

Ocorre um **MISS** quando a posição indicada pelo hash não contém um fluxo válido.

Nesse caso, o Controller deverá criar uma nova entrada.

Para um pacote com:

- Flow Key = nova chave;
- packet_length = L;
- timestamp = T;

a entrada será inicializada como:

Campo	Valor
valid	1
flow_key	Flow Key recebida
packet_count	1
byte_count	L
first_timestamp	T
last_timestamp	T

Após a escrita da entrada, o Controller poderá aceitar o próximo pacote.

14. Operação HIT

Ocorre um **HIT** quando a posição encontrada está válida e sua Flow Key é igual à Flow Key do pacote recebido.

A condição pode ser representada conceitualmente como:

HIT = valid AND stored_flow_key == incoming_flow_key

Nesse caso, não é criada uma nova entrada. O estado atual do fluxo é encaminhado para a **Feature Update Unit**, que realizará a atualização das estatísticas.

15. Operação COLLISION

Ocorre uma **COLLISION** quando a posição determinada pelo hash já está ocupada, porém a Flow Key armazenada é diferente da Flow Key recebida.

Conceitualmente:

COLLISION = valid AND stored_flow_key ≠ incoming_flow_key

Na primeira versão da arquitetura, a colisão será apenas sinalizada e o pacote não substituirá a entrada existente.

Essa simplificação é proposital. Estratégias mais complexas, como múltiplas posições, encadeamento ou Cuckoo Hashing, podem ser estudadas posteriormente. Sha et al., por exemplo, empregam Cuckoo Hashing em sua arquitetura de monitoramento de fluxos de alta velocidade.

Assim, a primeira implementação permite validar a arquitetura principal antes de adicionar mecanismos mais complexos de tratamento de colisões.

16. Feature Update Unit

A **Feature Update Unit** será responsável por atualizar os valores armazenados quando ocorre um HIT.

A operação básica será:

Feature	Operação
packet_count	packet_count + 1
byte_count	byte_count + packet_length
first_timestamp	mantém o valor
last_timestamp	recebe o timestamp atual

Por exemplo, considerando que a entrada contenha:

packet_count = 10

byte_count = 5000

e o novo pacote tenha:

packet_length = 700

então o novo estado será:

packet_count = 11

byte_count = 5700

O first_timestamp permanece igual e o last_timestamp passa a representar o timestamp do pacote recebido.

A atualização é realizada de forma incremental, evitando a necessidade de armazenar novamente todos os pacotes pertencentes ao fluxo.

Essa estratégia de manter informações estatísticas acumuladas por fluxo é utilizada em arquiteturas de monitoramento de tráfego em FPGA, incluindo o trabalho de Sha et al.

17. Flow Finalizer

O **Flow Finalizer** será responsável por transformar o estado acumulado de cada fluxo nas features finais.

Na primeira versão do projeto, a finalização ocorrerá quando o processamento do PCAP for concluído.

Quando o sinal pcap_done for recebido:

1. o Controller interrompe temporariamente a entrada de novos pacotes;
2. a Flow Table passa a ser percorrida sequencialmente;
3. entradas com valid = 0 são ignoradas;
4. entradas válidas são encaminhadas ao Flow Finalizer;
5. o Finalizer calcula as features derivadas;
6. o vetor final de features é disponibilizado na saída.

Nesta primeira versão não será utilizado timeout de fluxo nem detecção de FIN/RST do TCP para determinar o encerramento. Essa funcionalidade poderá ser adicionada em uma etapa futura.

18. Cálculo da Duration

A duration não precisa ser armazenada na Flow Table.

Ela pode ser calculada no momento da finalização utilizando:

duration = last_timestamp – first_timestamp

Essa escolha reduz a quantidade de informação armazenada durante o processamento e utiliza diretamente os timestamps que já são necessários para representar o início e o fim do fluxo.

Portanto, para a primeira versão:

Feature final	Origem
Packet Count	Flow Table
Byte Count	Flow Table
Duration	last_timestamp - first_timestamp

Caso sejam utilizadas somente essas três features, o vetor final possuirá:

$$24 + 40 + 64 = 128 \text{ bits}$$

19. Comunicação entre os módulos

A comunicação entre os módulos será baseada no protocolo **valid/ready**.

O princípio é:

transferência = valid AND ready

O módulo produtor coloca os dados na interface e ativa valid. O consumidor ativa ready quando estiver preparado para receber os dados. A transferência acontece somente quando os dois sinais estiverem ativos simultaneamente.

O produtor deve manter valid e os dados estáveis enquanto estiver aguardando ready.

Esse tipo de mecanismo é amplamente utilizado em arquiteturas de processamento de pacotes em hardware, permitindo controlar o fluxo de dados e inserir espera quando determinado módulo ainda estiver ocupado.

20. Controle de backpressure

O controle de **backpressure** será necessário porque o processamento de um pacote pode exigir várias etapas internas, como:

1. receber metadados;
2. calcular índice;
3. realizar leitura da Flow Table;
4. comparar a Flow Key;
5. atualizar ou criar a entrada;
6. escrever o resultado.

Durante uma operação pendente, o Controller poderá manter sua entrada desabilitada, evitando que um novo pacote seja aceito antes que o processamento anterior seja concluído.

Isso é importante principalmente para evitar situações em que dois pacotes consecutivos pertencentes ao mesmo fluxo tentem atualizar simultaneamente a mesma entrada.

Por exemplo, se dois pacotes forem recebidos muito próximos um do outro, ambos não devem ler o mesmo packet_count antigo e posteriormente sobrescrever a tabela com resultados incorretos.

Assim, inicialmente será adotada uma estratégia conservadora:

um pacote é aceito → operação é concluída → próximo pacote é aceito.

Posteriormente, caso os resultados de desempenho indiquem necessidade, essa restrição poderá ser reduzida por meio de pipeline ou outras técnicas de paralelização.

21. Sinais de controle

A interface entre os módulos deverá possuir sinais de dados e controle suficientes para indicar o estado da operação.

Uma estrutura inicial pode ser:

Sinal	Função
valid	Indica que os dados de saída são válidos
ready	Indica que o módulo está pronto para receber
flow_key	Chave do fluxo
packet_length	Tamanho do pacote atual
timestamp	Timestamp do pacote
hash_index	Posição indicada pela função hash
hit	Indica ocorrência de HIT
miss	Indica ocorrência de MISS
collision	Indica ocorrência de colisão
update_done	Indica conclusão da atualização
pcap_done	Indica fim do processamento do PCAP
final_valid	Indica vetor final disponível

Os nomes podem ser ajustados durante a implementação em Verilog/SystemVerilog; neste momento, o objetivo é definir claramente o comportamento esperado das interfaces.

22. Sequência completa de processamento

Considerando um pacote recebido pelo sistema, o processamento completo será:

Recepção do pacote → Parsing → Formação da Flow Key → Hash → Consulta da Flow Table → HIT/MISS/COLLISION → Atualização ou criação da entrada → Próximo pacote

Enquanto o PCAP estiver sendo processado, a Flow Table mantém o estado acumulado dos fluxos.

Ao final:

PCAP concluído → Varredura da Flow Table → Flow Finalizer → Cálculo das features → Vetor final

Dessa forma, o hardware não precisa gerar o vetor de features a cada pacote. Durante o processamento, ele mantém apenas o estado necessário para cada fluxo e gera as features finais quando o fluxo é finalizado.

23. Critérios de aceitação

A implementação será considerada funcional quando os seguintes comportamentos forem validados:

Teste	Resultado esperado
Primeiro pacote de um fluxo	Criar entrada com packet_count = 1
Segundo pacote do mesmo fluxo	Gerar HIT e incrementar packet_count
Novo pacote com tamanho conhecido	Incrementar corretamente byte_count
Pacotes consecutivos	Não perder nem sobrescrever atualizações
Dois fluxos com hashes diferentes	Criar entradas independentes
Dois fluxos com mesmo índice	Detectar COLLISION
Finalização do PCAP	Percorrer a tabela e gerar os vetores
Duration	last_timestamp - first_timestamp
Entrada inválida	Não gerar vetor de features

Além da verificação funcional, posteriormente será possível avaliar aspectos de implementação na FPGA, como utilização de LUTs, registradores, BRAM, frequência máxima, latência e throughput.

24. Resumo dos parâmetros definidos

Parâmetro	Valor inicial
Protocolo de rede	IPv4
Protocolos de transporte	TCP/UDP
Identificação	5-tupla
Flow Key	104 bits
Packet Length	16 bits
Timestamp	64 bits
Packet Count	24 bits
Byte Count	40 bits
Flow Table	32 entradas
Hash Index	5 bits
Registro da Flow Table	297 bits
Features principais	Packet Count, Byte Count, Duration
Duration	last_timestamp - first_timestamp
Finalização	Fim do PCAP
Timeout	Não implementado inicialmente
Tratamento de colisão	Sinalização, sem substituição
Direção	Uniflow
Protocolo de interface	Valid/Ready

Referências utilizadas

[1] SHA, G.; GUO, Y.; WANG, Z.; ZENG, Y. *A High-Performance and Accurate FPGA-Based Flow Monitor for 100 Gbps Networks*. Electronics, 2022. É a principal referência para a estrutura da Flow Table, identificação por 5-tupla, armazenamento de timestamps, contagem de pacotes, tamanho acumulado e atualização do estado do fluxo.

<https://www.mdpi.com/2079-9292/11/13/1976>

[2] KEKELY, M.; PUŠ, V.; KOŘENEK, J. *Low-Latency Modular Packet Header Parser for FPGA*. ANCS, 2012. Referência para a abordagem modular de parsing de cabeçalhos de pacotes em FPGA.

https://www.fit.vut.cz/research/result-file/c97063/278875/korenek_12_Q4_01.pdf

[3] SCHUEHLER, D.; LOCKWOOD, J. W. *A Modular System for FPGA-Based TCP Flow Processing in High-Speed Networks*. FPL, 2004. Referência para processamento e acompanhamento de fluxos TCP em FPGA, especialmente no gerenciamento de estado associado aos fluxos.

https://www.researchgate.net/publication/220759341_A_Modular_System_for_FPGA-Based_TCP_Flow_Processing_in_High-Speed_Networks

[4] NGUYEN, H.; ZAMBRENO, J.; MEMIK, G. *Flow Monitoring in High-Speed Networks with 2D Hash Tables*. FPL, 2004. Referência para utilização de estruturas hash para monitoramento de fluxos e armazenamento de informações de fluxo em FPGA.

<https://users.ece.northwestern.edu/~choudhar/Publications/NguZam04A.pdf>